

Cryochamber

AI 智能体的冬眠调度器

产品使用指南

面向运维者、开发者及感兴趣的读者

目录

1. Cryochamber 是什么?	3
2. 核心概念	3
2.1. 会话 (Session)	3
2.2. 守护进程 (Daemon)	3
2.3. IPC 通信	3
2.4. TODO 列表	3
2.5. 收件箱 / 发件箱 (Inbox / Outbox)	3
3. 系统架构	4
4. 会话生命周期	5
5. 快速上手	5
5.1. 安装	5
5.2. 创建项目	5
5.3. 启动守护进程	6
5.4. 监控进度	6
5.5. 发送与接收消息	6
5.6. 停止	6
6. 智能体协议	6
7. 人类 ↔ 智能体通信	7
7.1. 直接消息 (本地)	7
7.2. GitHub Discussions 同步	7
7.3. Zulip 同步	8
8. 配置参考	8
9. 运行时文件参考	8
10. 完整示例演练	9
11. Provider 轮换	10
12. 开发者指南	10

1. Cryochamber 是什么？

Cryochamber（冰室）是一个用 Rust 编写的开源 AI 智能体调度器。它解决了一个根本性问题：大型任务的执行时间往往超过单次 LLM 上下文窗口的承载能力。

现代 AI 智能体（Claude、OpenCode、GPT-4……）能写代码、浏览网页、开发软件——但它们会超时、触达速率限制，或者需要暂停、隔夜继续。Cryochamber 为它们提供了一个持久化的运行框架：

⚠ 问题所在

- 智能体在任务中途超时
- 两次运行之间没有记忆
- 需要人工手动重启
- API 速率限制阻断夜间工作

✓ Cryochamber 提供的能力

- 通过 TODO 列表实现定时唤醒
- 跨会话的持久化日志与备注
- 人类与智能体之间的消息通信
- 崩溃后自动重试

这个名字来自一个比喻：智能体在两次会话之间被放入冰室（cryochamber）——状态被冻结，资源被释放，并在恰当的時刻被唤醒。

2. 核心概念

2.1. 会话（Session）

会话是智能体进程的一次运行。每个会话都有编号、任务描述、开始时间戳和结束结果（complete、exit <N>、timeout……）。会话在 `cryo.log` 中以 `--- CRYO SESSION N --- / --- CRYO END ---` 标记分隔。

2.2. 守护进程（Daemon）

守护进程是一个由操作系统服务层管理的长期后台进程（macOS 用 `launchd`、Linux 用 `systemd`、Windows 用 `SCM`）。它负责：

- 等待直到最早的待办 TODO 时间；
- 为每个会话启动智能体子进程；
- 强制执行可配置的会话超时；
- 以指数退避策略重试崩溃的智能体（5 秒 → 15 秒 → 60 秒）；
- 监听 `messages/inbox/` 以实现响应式唤醒。

2.3. IPC 通信

智能体子进程通过平台 IPC 通道（Unix 域套接字 / Windows 命名管道）与守护进程通信，使用 `cryo-agent` CLI。这使智能体完全解耦——它只需执行 `shell` 命令即可。

2.4. TODO 列表

智能体通过 `todo.json` 控制自己的唤醒调度。添加一个带有未来 `--at` 时间的 TODO，守护进程就会休眠到该时间后重新启动智能体。若没有待处理的 TODO，守护进程将一直空闲。

2.5. 收件箱 / 发件箱（Inbox / Outbox）

`messages/` 目录中有一个简单的基于文件的消息总线。人类通过 `cryo send` 向 `inbox/` 写入消息；智能体在会话开始时读取这些消息。智能体将回复写入 `outbox/`；人类通过 `cryo receive` 读取。GitHub Discussions 和 Zulip 等渠道可以将此消息总线镜像到互联网上。

3. 系统架构

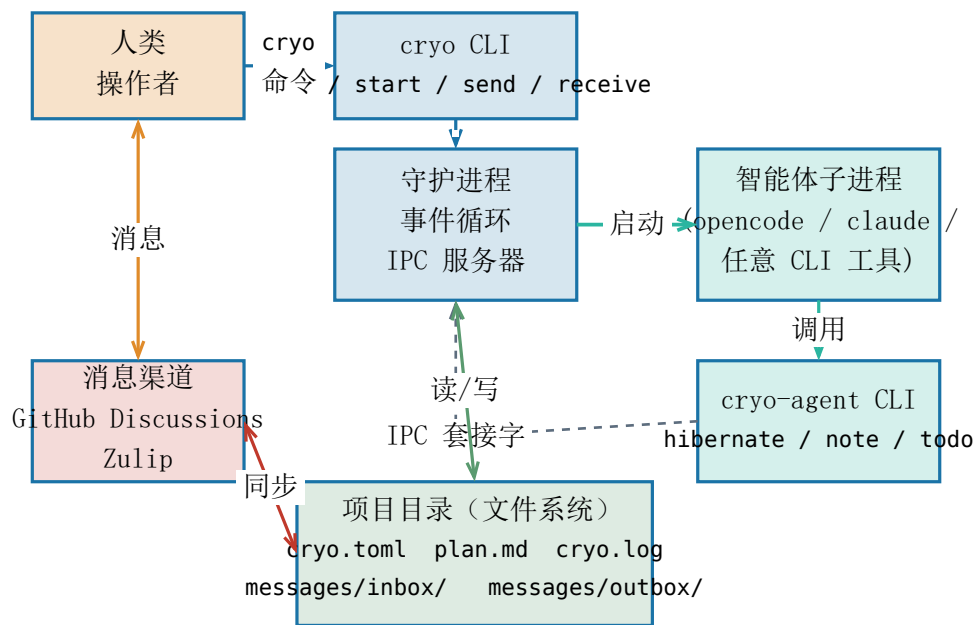
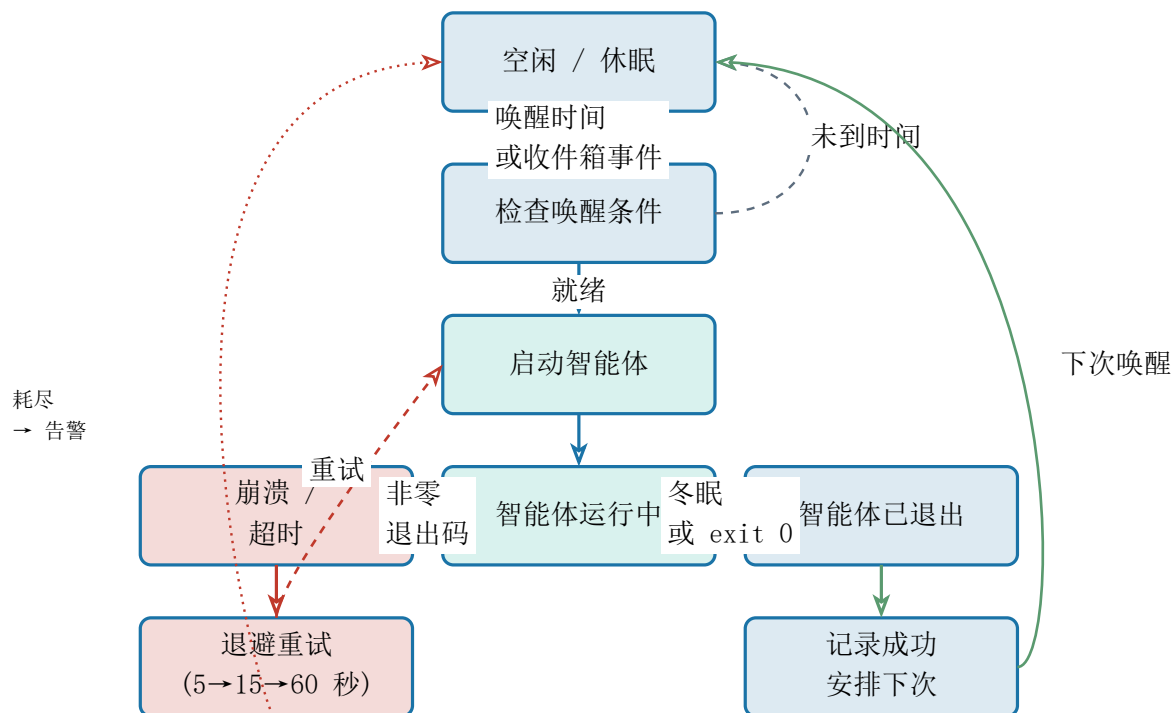


Figure 1: Cryochamber 组件架构图。

该图展示了五个主要参与者：

参与者	职责
cryo CLI	操作者的入口点——初始化项目、启动/停止守护进程、查看日志、发送消息。
守护进程	长期后台进程。管理会话生命周期、IPC 服务器和唤醒逻辑。
智能体子进程	实际运行的 AI 工具（如 opencode run）。完全可替换——任何 CLI 程序均可。
cryo-agent CLI	智能体的工具箱。通过 IPC 发送命令：冬眠、添加备注、安排待办、发送消息。
消息渠道	可选的桥接器（GitHub Discussions、Zulip），将收件箱/发件箱镜像到互联网服务。

4. 会话生命周期



守护进程无限循环执行。关键行为：

- 正常退出：智能体调用 `cryo-agent hibernate`；守护进程记录会话并从 `todo.json` 安排下次唤醒。
- 崩溃/超时：守护进程以指数退避策略重试。在达到 `max_retries` 次连续失败后，触发备用告警（死亡开关）写入发件箱。
- 响应式唤醒：`messages/inbox/` 中出现新文件，或执行 `cryo wake` 命令，均可提前结束休眠。

5. 快速上手

5.1. 安装

```
cargo install cryochamber          # 从 crates.io 安装
# 或者从源码安装：
git clone https://github.com/GigggleLiu/cryochamber
cd cryochamber && cargo install --path .
```

安装完成后会得到四个可执行文件：`cryo`、`cryo-agent`、`cryo-gh`、`cryo-zulip`。

5.2. 创建项目

```
mkdir my-ai-project && cd my-ai-project
echo "# 计划\n在此编写任务计划。" > plan.md
cryo init --agent "opencode run"
```

`cryo init` 会创建：

- `cryo.toml` — 项目配置文件
- `plan.md` — （已存在）每次会话开始时由智能体读取
- `CLAUDE.md` / `AGENTS.md` — 注入智能体上下文的 `Cryochamber` 协议

5.3. 启动守护进程

```
cryo start          # 使用 cryo.toml 中的智能体
cryo start --agent "claude -p"  # 临时覆盖智能体命令
```

在 macOS/Linux 上，这会安装一个能在重启后继续运行的 launchd / systemd 服务。设置 CRYO_NO_SERVICE=1 可跳过服务安装，以普通后台进程方式运行。

5.4. 监控进度

```
cryo status      # 守护进程状态、会话编号、下次唤醒时间
cryo watch       # 实时跟踪 cryo.log (类似 tail -f)
cryo watch --all  # 从日志开头显示
cryo log         # 输出完整结构化日志
```

5.5. 发送与接收消息

```
cryo send "请在下次会话前审查 PR"
cryo receive          # 从发件箱读取智能体的回复
```

cryo send 会在 messages/inbox/ 中放置一条 Markdown 消息。守护进程会唤醒智能体（如果正在休眠），智能体在下次会话开始时读取该消息。

5.6. 停止

```
cryo cancel      # 停止守护进程并移除服务
cryo clean --force  # 停止 + 删除所有运行时文件
```

6. 智能体协议

守护进程启动智能体时，会在提示词顶部注入一个 Cryochamber 协议块。该块指导智能体使用 cryo-agent 子命令进行结构化通信。

📄 会话 workflow（智能体视角）

- 第一步 — 定向：读取 plan.md，执行 cryo-agent todo list，检查收件箱。
- 第二步 — 工作：完成任务。用 cryo-agent reply 回复收件箱消息。
- 第三步 — 记录：cryo-agent note "完成了什么，下一步是什么"。
- 第四步 — 调度：cryo-agent todo add "下一个任务" --at <时间>，时间用 cryo-agent time "+2 hours" 计算。
- 第五步 — 冬眠：cryo-agent hibernate --summary "..."（必须是最后一条命令）。

cryo-agent 命令	功能说明
hibernate --summary "..."	结束会话。守护进程记录结果并安排下次唤醒。
hibernate --complete	标记整个项目已完成。
hibernate --exit 1 --summary "..."	标记失败 / 阻塞状态。
note "文本"	向会话日志追加一条备注。下次会话可通过 parse_latest_session_notes 读取。
send "消息"	向 messages/outbox/ 写入一条消息。

reply "消息"	回复当前收件箱消息。
receive	读取 messages/inbox/（本地操作，无需守护进程）。
alert <动作> <目标> "消息"	设置死亡开关：若智能体未按时唤醒，则对 <目标> 触发 <动作>。
todo add "任务" --at <ISO>	添加带时间的待办事项。守护进程在最早的待办时间唤醒。
todo list	列出所有待处理的待办事项。
todo done <id>	将某个待办标记为已完成。
time	打印当前 ISO8601 本地时间。
time "+2 hours"	计算并打印未来某时刻的 ISO8601 时间。

7. 人类 ↔ 智能体通信

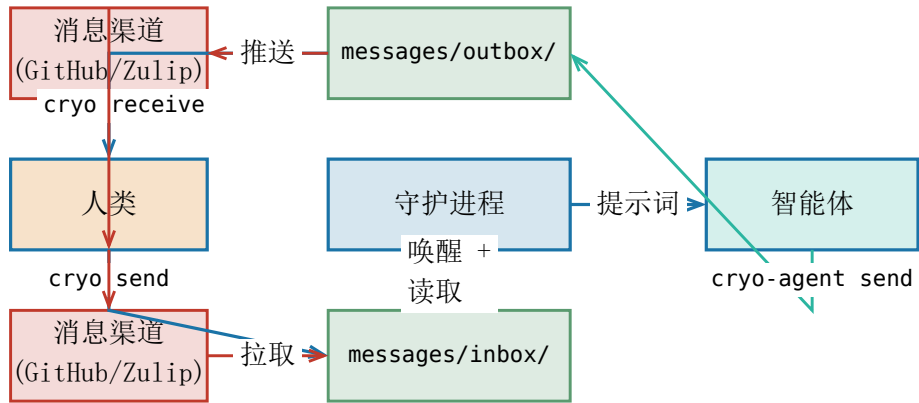


Figure 3: 人类 ↔ 智能体通信流程图。

7.1. 直接消息（本地）

最简单的方式——无需任何外部账号：

```
# 人类发送消息
cryo send "请为认证模块添加单元测试"

# 人类读取智能体的回复
cryo receive
```

7.2. GitHub Discussions 同步

```
cryo-gh init --repo owner/repo --discussion 42
cryo-gh sync           # 启动后台同步守护进程
cryo-gh status         # 查看同步配置
cryo-gh unsync         # 停止后台同步
```

GitHub Discussion 中的新评论在每个同步周期被拉取到 messages/inbox/。会话摘要则作为新评论推送回去。

7.3. Zulip 同步

```
cryo-zulip init --config ~/.zuliprc --stream "dev"
cryo-zulip sync      # 启动后台同步守护进程
cryo-zulip status    # 查看同步状态
```

在配置的 Zulip 流中发布的消息会出现在收件箱中；智能体的回复则被推送回该流。

8. 配置参考

项目配置保存在 cryo.toml 中：

```
agent = "opencode run"           # 默认智能体命令
max_retries = 3                   # 触发备用告警前的崩溃重试次数
max_session_duration = 1800      # 会话超时时间（秒，0 表示不限制）
watch_inbox = true               # 有新收件箱消息时通过 inotify 响应式唤醒
fallback_alert = "outbox"        # "outbox" | "notify" | "none"
report_interval_hours = 24       # 0 = 禁用周期性报告
report_time = "09:00"           # 每日报告的时间点

# 配置多个 Provider 实现自动轮换：
[[providers]]
name = "primary"
env = { OPENAI_API_KEY = "sk-..." }

[[providers]]
name = "fallback"
env = { ANTHROPIC_API_KEY = "sk-ant-..." }
```

配置项	默认值	说明
agent	opencode run	启动智能体子进程的 Shell 命令。
max_retries	3	连续崩溃后放弃并发送备用告警前的重试次数。
max_session_duration	0	每次会话的最大秒数。0 表示不限制。
watch_inbox	true	使用文件系统事件，在新收件箱消息到达时立即唤醒守护进程。
fallback_alert	"outbox"	重试耗尽时的处理方式：写入发件箱、发送桌面通知或不做任何操作。
report_interval_hours	0	周期性会话统计桌面通知的间隔。0 表示禁用。
report_time	"09:00"	每日报告对齐的时钟时间。
providers	[]	AI Provider 配置列表，失败时轮换。每条记录可设置环境变量。

9. 运行时文件参考

文件 / 目录	用途
cryo.toml	项目配置。手动编辑。由 cryo init 创建。
plan.md	每次智能体会话中注入的任务描述。

CLAUDE.md / AGENTS.md	面向智能体的 Cryochamber 协议（由 <code>cryo init</code> 自动生成）。
timer.json	运行时状态：会话编号、PID 锁、重试计数、CLI 覆盖值。由守护进程写入。
cryo.log	仅追加的结构化事件日志。可通过 <code>cryo log</code> / <code>cryo watch</code> 读取。
cryo-agent.log	智能体子进程的原始标准输出/标准错误。
todo.json	待处理的待办事项。驱动唤醒调度。
messages/inbox/	智能体的收件消息。由 <code>cryo send</code> 和渠道同步写入。
messages/outbox/	智能体的回复。由 <code>cryo receive</code> 读取，并由渠道同步推送。
messages/inbox/archive/	已处理（已读）收件箱消息的归档。
.cryo/cryo.sock	智能体 ↔ 守护进程 IPC 的 Unix 域套接字（仅 Unix 系统）。
gh-sync.json	GitHub Discussion 同步状态（最后游标、已推送会话）。由 <code>cryo-gh init</code> 创建。
zulip-sync.json	Zulip 同步状态（最后消息 ID、已推送会话）。由 <code>cryo-zulip init</code> 创建。

10. 完整示例演练

以下追踪一个长期编码任务的完整一个轮次。

1 操作者初始化项目

```
mkdir chess-engine && cd chess-engine
cat > plan.md << 'EOF'
# 象棋引擎
用 Rust 实现一个兼容 UCI 协议的象棋引擎。
本次会话聚焦于 alpha-beta 剪枝。
EOF
cryo init --agent "opencode run"
cryo start
```

守护进程启动；智能体立即唤醒（会话 1）。

2 智能体工作（会话 1）

```
# 在智能体进程内部 — 由 LLM 发出的 cryo-agent 命令
cryo-agent todo add "实现走法生成" --at 2026-03-07T09:00
cryo-agent note "棋盘表示已完成。下一步：走法生成。"
cryo-agent hibernate --summary "Board 结构体已完成。已安排明天的走法生成任务。"
```

守护进程记录 --- CRYO END ---，读取 TODO，并休眠至 09:00。

3 人类在休眠期间发送消息

```
cryo send "另外请加上 perft 测试方便调试"
```

守护进程的收件箱监听器触发，提前唤醒智能体（会话 2 开始）。

4 操作者检查进度

```
cryo status
# → 会话: 3 | 下次唤醒: 2026-03-08T09:00 | 智能体: opencode run
cryo receive
# → 智能体消息: "走法生成已完成。Perft(5) = 4865609 ✓"
```

11. Provider 轮换

当 `cryo.toml` 中定义了多个 `[[providers]]` 时，守护进程会在连续失败后轮换使用它们。适用场景包括：

- 速率限制容错（轮换到不同的 API Key 或模型）
- 多模型实验（失败时从 GPT-4 切换到 Claude）
- 预算控制（优先用低成本模型，失败后切到高成本模型）

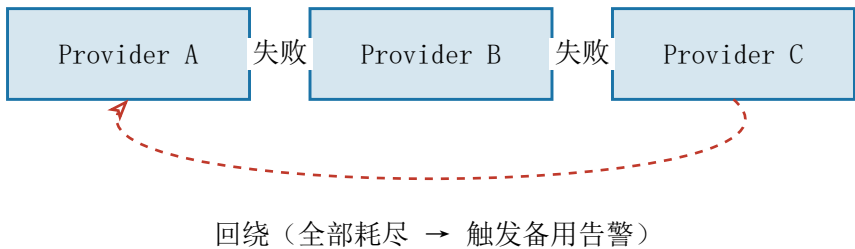


Figure 4: 连续失败时的 Provider 轮换顺序。

12. 开发者指南

🔧 开发工作流

```
# 运行所有测试
cargo test

# 代码检查 (CI 中警告即错误)
cargo clippy --all-targets -- -D warnings

# 测量覆盖率
make coverage

# 端到端运行内置的 mr-lazy 示例
make example DIR=examples/mr-lazy
make example-cancel DIR=examples/mr-lazy
```

贡献者的关键入口点：

文件	如果你想……请从这里开始
<code>src/daemon.rs</code>	理解或修改会话事件循环、重试逻辑、唤醒检测。
<code>src/bin/cryo.rs</code>	添加或修改面向操作者的 CLI 命令。
<code>src/bin/cryo_agent.rs</code>	添加或修改面向智能体的 IPC 命令。

<code>src/platform/</code>	移植到新操作系统或修复平台特定行为。
<code>src/channel/zulip.rs</code>	理解 Zulip HTTP 客户端或添加新的消息渠道。
<code>src/agent.rs</code>	修改智能体提示词的构建方式或子进程的启动方式。
<code>templates/</code>	编辑 <code>cryo init</code> 注入的默认协议、计划或配置模板。
<code>tests/</code>	添加集成测试。CLI 测试使用 <code>assert_cmd</code> ；HTTP 测试使用内联最小化 <code>mock</code> 服务器。

Cryochamber 采用 MIT 许可证。源码：<https://github.com/GiggleLiu/cryochamber>
文档：<https://giggleliu.github.io/cryochamber>